

TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
NATIONAL COLLEGE OF ENGINEERING

A
MINOR PROJECT FINAL REPORT
ON
“GESTURE BASED VOICE COMMUNICATION SYSTEM”

SUBMITTED BY:

MOBIN KOIRALA (NCE080BEI024)
SHRAWAN PRASAI (NCE080BEI038)
SOHAM KC (NCE080BEI040)
SUDARSHAN DANGAL (NCE080BEI043)

SUBMITTED TO:

DEPARTMENT OF ELECTRONICS & COMPUTER ENGINEERING

LALITPUR, NEPAL

AUGUST 25, 2026

Page of Approval

TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
NATIONAL COLLEGE OF ENGINEERING
DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

The undersigned certifies that they have read and recommended to the Institute of Engineering for acceptance of a project report entitled “**Gesture Based Voice Communication System**” submitted by **Mobin Koirala, Shrawan Prasai, Soham KC, and Sudarshan Dangal** in partial fulfillment of the requirements for the Bachelor’s degree in Electronics and Computer Engineering.

.....

Supervisor

Er. Sagar Panta

Assistant Lecturer

Department of Electronics and Computer Engineering,

National College of Engineering, IOE, TU.

Date of approval: **August 25, 2026**

Copyright

The authors have agreed that the Library, Department of Electronics and Computer Engineering, National College of Engineering and Institute of Engineering may make this report freely available for inspection. The authors further agree that permission for extensive copying of this project report for scholarly purposes may be granted by the supervisors who supervised the work recorded herein or, in their absence, by the Head of the Department wherein the project report was done. It is understood that recognition will be given to the authors of this report and to the Department of Electronics and Computer Engineering, National College of Engineering, IOE for any use of the material of this report. Copying, publication, or other use of this report for financial gain without approval of the Department of Electronics and Computer Engineering, National College of Engineering, Institute of Engineering, and the authors' written permission is prohibited.

Request for permission to copy or to make any other use of the material in this report in whole or in part should be addressed to:

Er. Suroj Burlakoti

Head of the Department

Department of Electronics and Computer Engineering

National College of Engineering

Talchhikhel, Lalitpur.

Acknowledgments

We would like to express our sincere gratitude to our project supervisor, **Er. Sagar Panta**, for continuous guidance, constructive feedback, technical suggestions, and encouragement throughout the development of this project. We are also thankful to the Department of Electronics and Computer Engineering, National College of Engineering, Institute of Engineering, Tribhuvan University, for providing the academic environment and facilities required to complete this work.

We extend our appreciation to our teachers, laboratory staff, friends, and all those who supported us during hardware assembly, sensor data collection, machine learning development, testing, and documentation. Finally, we would like to thank our families for their constant encouragement and support.

Sincerely,

Mobin Koirala

Shrawan Prasai

Soham KC

Sudarshan Dangal

Abstract

This project presents a wearable gesture-based voice communication system that converts hand gestures into audible output using a sensor-equipped glove. The system uses five flex sensors, six copper-tape touch detectors, an MPU6050 providing three accelerometer and three gyroscope parameters, an ESP32, and a DFPlayer Mini with a 3 W speaker. The five flex sensors capture finger bending, while the six touch detectors provide additional contact information used directly as machine-learning features.

The system supports 26 American Sign Language (ASL) alphabet gestures and twelve pre-defined shortcut mappings for commonly used voice outputs. The complete gesture input contains 17 active parameters: five flex values, six MPU6050 values, and six touch values. Sensor data are calibrated, normalized, labelled, and used to train a Random Forest classifier for embedded gesture recognition. The recognized class is mapped to a prerecorded audio file and played locally through the DFPlayer Mini. The project demonstrates a approach to combining finger position, touch relationships, and hand motion for gesture-to-voice communication.

Keywords: *American Sign Language, smart glove, gesture recognition, Random Forest, ESP32, assistive communication*

Contents

Page of Approval	ii
Copyright	iii
Acknowledgments	iv
Abstract	v
Contents	viii
List of Figures	ix
List of Tables	x
List of Abbreviations	xi
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Objectives	2
1.4 Scope	3
2 Literature Review	4
2.1 Related work	4
2.2 Related theory	4
2.2.1 Flex Sensor	4
2.2.2 MPU6050 Inertial Measurement Unit	5
2.2.3 DFPlayer Mini	5
2.2.4 Copper-Tape Touch Detector (Touch Sensor)	6
2.2.5 3 W Speaker	6
2.2.6 ESP32 Microcontroller	7
2.2.7 UART Communication	7
2.2.8 Random Forest for Gesture Recognition	8
2.2.9 Gesture-to-Voice Conversion	8
3 Methodology	9
3.1 Data Collection	9

3.2	Data Preprocessing	9
3.2.1	Data Cleaning and Validation	10
3.2.2	Sensor Normalization	10
3.2.3	Final 17-Parameter Feature Vector	10
3.2.4	Sensor Parameter and Class Mapping	11
3.3	Dataset Preparation	12
3.3.1	Session Data Merging	12
3.3.2	Final Dataset	12
3.4	Random Forest Model Training and Evaluation	12
3.5	Embedded Implementation	13
3.5.1	Model Export	13
3.5.2	ESP32 Integration	14
3.5.3	Real-Time Gesture Inference	14
3.6	Operating Modes and Audio Output	14
3.6.1	Alphabet Mode and Mapping	14
3.6.2	Word Mode Gesture Mapping	16
3.6.3	Audio Output	17
3.7	Development and Deployment Workflow	18
4	Hardware Configuration / Experimental Setup	19
4.1	Hardware Components	19
4.2	Hardware Assembly	19
4.2.1	Circuit and Power Configuration	21
4.3	Experimental Setup	21
4.4	Experimental Procedure	22
5	System design	24
5.1	System Architecture	24
5.1.1	Block Diagram Description	24
5.2	System Operation Flowchart	26
6	Results & Discussion	28
6.1	Experiment Setup	28
6.2	Software Development Approach	28
6.3	Model Deployment	29
6.4	Testing/evaluation/validation	30
6.4.1	Model Performance	30
6.4.2	Random Forest Feature Importance	31
6.4.3	Training Behaviour	31
6.4.4	Per-Class Performance	32

6.4.5	Confusion Matrix Analysis	33
6.4.6	Discussion	34
7	Conclusions	35
8	Limitations and Future Enhancement	36
8.1	Limitations	36
8.2	Future Enhancements	36
	References	36
	Appendix	39
	Additional Hardware Photographs	39

List of Figures

2.1	Flex sensor used in the smart glove.	5
2.2	MPU6050 inertial measurement unit.	5
2.3	DFPlayer Mini audio module.	6
2.4	Copper-Tape touch Detector.	6
2.5	Speaker for audio output.	7
2.6	ESP32 DevKit.	7
3.1	Dataset collection and sensor readings.	9
3.2	Data training and model development pipeline.	12
3.3	ASL alphabet Categorization.	15
4.1	Voltage-divider circuit for the flex sensor.	20
4.2	Final circuit prototype.	20
4.3	Final hardware circuit prototype.	21
4.4	Hardware assembly of the glove system.	21
5.1	Overall system block diagram.	24
5.2	Overall gesture recognition and audio output flowchart.	26
6.1	Overall performance of the Random Forest model on the held-out session. . . .	30
6.2	Feature importance of the 17 sensor parameters in the Random Forest model. .	31
6.3	Training and held-out session accuracy for different Random Forest sizes. . . .	32
6.4	Precision, recall, and F1-score for individual alphabet classes.	32
6.5	Confusion matrix of the Random Forest classifier on the held-out session. . . .	33
8.1	Additional prototype photographs.	39
8.2	Additional hardware assembly photographs.	39

List of Tables

3.1	Final Sensor-Fusion Feature Vector	11
3.2	Final Sensor-Location and Dataset-Parameter Mapping	11
3.3	ASL Alphabet Gesture-to-Audio Mapping	16
3.4	Word Mode Gesture-to-Audio Mapping	17

List of Abbreviations

ADC	Analog-to-Digital Converter
AI	Artificial Intelligence
ASL	American Sign Language
CSV	Comma-Separated Values
GPIO	General-Purpose Input/Output
I2C	Inter-Integrated Circuit
IMU	Inertial Measurement Unit
JSON	JavaScript Object Notation
ML	Machine Learning
MP3	MPEG-1 Audio Layer III
RF	Random Forest
SCL	Serial Clock Line
SDA	Serial Data Line
UART	Universal Asynchronous Receiver-Transmitter

1. Introduction

1.1 Background

Communication is a fundamental part of daily life, yet people with speech and hearing impairments can face significant barriers when those around them do not understand sign language. Basic activities such as requesting assistance, expressing discomfort, greeting another person, or asking for an object can become difficult when an interpreter is not available. Conventional methods such as writing, communication boards, or dependence on interpreters can reduce the speed and independence of spontaneous communication [1][2].

Wearable smart gloves provide a promising approach to reducing this communication barrier. Sensors mounted on a glove can measure finger bending and hand motion, while a microcontroller can classify the resulting sensor pattern as a gesture. The recognized gesture can then be mapped to a letter, word, or phrase and converted into an audible message. Previous studies have demonstrated the usefulness of combining flex sensors and inertial sensors for gesture recognition [3] [4] [5].

This project develops a low-cost gesture-based voice communication glove using an ESP32, flex sensors, an MPU6050 IMU, and a DFPlayer Mini. The system is designed as a hybrid platform with an Alphabet Mode for ASL finger spelling and a Word Mode for frequently used phrases. The primary voice output is generated locally through prerecorded audio stored on the DFPlayer Mini, allowing the device to operate without a continuous internet connection or external computer.

1.2 Problem Statement

People who use sign language may face communication difficulties when the people around them do not understand sign language. Conventional methods such as writing or depending on an interpreter can reduce the speed and independence of communication. A wearable system should therefore recognize hand gestures from multiple sensor inputs and provide direct audible output without relying on an external computer or internet connection.

The project addresses the following problems:

- i. Difficulty in communicating with people who do not understand sign language.
- ii. Difficulty distinguishing gestures with similar finger positions using flex sensors alone.
- iii. Need for additional touch and hand-motion information to improve gesture discrimination.

- iv. Need for a portable system that supports both alphabet spelling and predefined shortcut gestures.

1.3 Objectives

The main objectives of this project are:

- i. To develop a glove that recognizes 26 ASL finger-spelled alphabets and twelve predefined shortcut gestures using five flex sensors, an MPU6050 IMU, and copper-tape-based finger touch detectors.
- ii. To provide real-time voice communication by playing prerecorded audio corresponding to recognized ASL gestures and predefined phrases.

1.4 Scope

The scope of this project includes:

- i. Designing a wearable glove that detects finger bending using five flex sensors, hand motion using an MPU6050, and finger contact using copper-tape touch sensors for gesture recognition.
- ii. Implementing a hybrid dual-mode gesture recognition system: Mode 1 for 26 ASL alphabet gestures enabling spelling of words through a letter buffer, and Mode 2 for twelve predefined shortcut gestures providing instant voice output.
- iii. To develop and deploy a lightweight Random Forest machine-learning model on the ESP32 for real-time, on-device gesture recognition.
- iv. Providing voice output through the DFPlayer Mini and speaker.

2. Literature Review

2.1 Related work

Wearable smart gloves have been investigated extensively for gesture recognition and assistive communication. Kurundkar et al. showed the value of combining flex sensors with an MPU6050-based inertial measurement unit, particularly where hand orientation or motion helps distinguish similar gestures [3]. Filipowska et al. similarly investigated machine-learning-based gesture recognition gloves and demonstrated the usefulness of multi-sensor input for classification [4].

Machine-learning methods have also been applied directly to sensor data collected from gloves. Lu et al. reported an AI-enabled gesture-language-recognition system using strain-sensor arrays, illustrating how sensor-based machine learning can provide real-time gesture feedback [6]. Other research has explored flexible sensing, deep learning, and smart textile gloves for recognizing complex hand movements [7] [8] [9].

ESP32-based embedded systems are attractive for this application because they combine processing capability, analog input, I2C, UART, and wireless communication in a relatively inexpensive microcontroller. Existing smart-glove work has demonstrated standalone processing and audio output using embedded controllers [10]. Pre-recorded audio is also a practical output method because it avoids the computational cost of real-time speech synthesis [1].

The present project combines these ideas into a single low-cost platform: sensor fusion for gesture recognition, on-device Random Forest inference, and local audio playback. The dual-mode architecture is intended to combine flexible letter spelling with rapid phrase selection.

2.2 Related theory

2.2.1 Flex Sensor

A flex sensor is a variable-resistance sensor whose resistance changes as the sensor bends. When mounted along a finger, the change in resistance can be converted into an analog voltage through a voltage-divider circuit. The resulting value provides an estimate of finger bending and forms a primary feature for gesture classification [3][4].

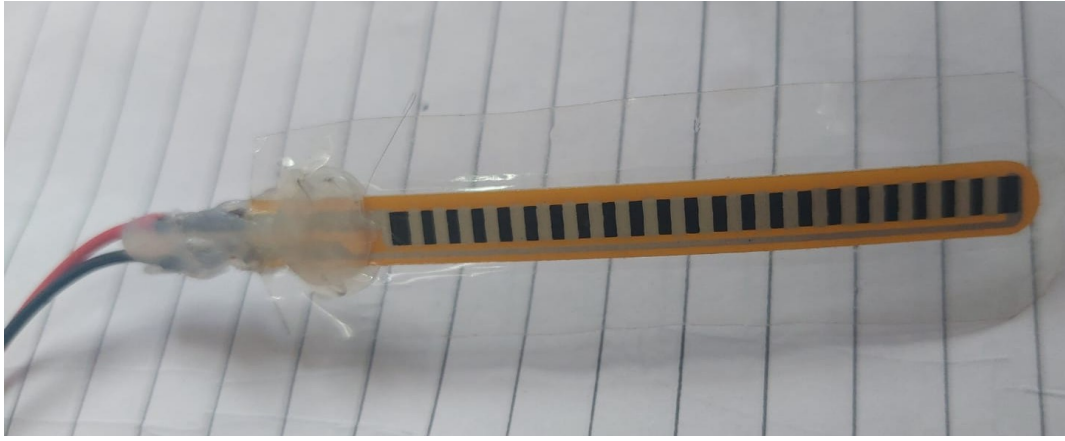


Figure 2.1: Flex sensor used in the smart glove.

2.2.2 MPU6050 Inertial Measurement Unit

The MPU6050 combines a three-axis accelerometer and a three-axis gyroscope. It communicates with the ESP32 through I2C and provides acceleration and angular-velocity measurements along the X, Y, and Z axes. These measurements provide information about hand orientation and movement, which can complement finger-bending measurements [11][5].

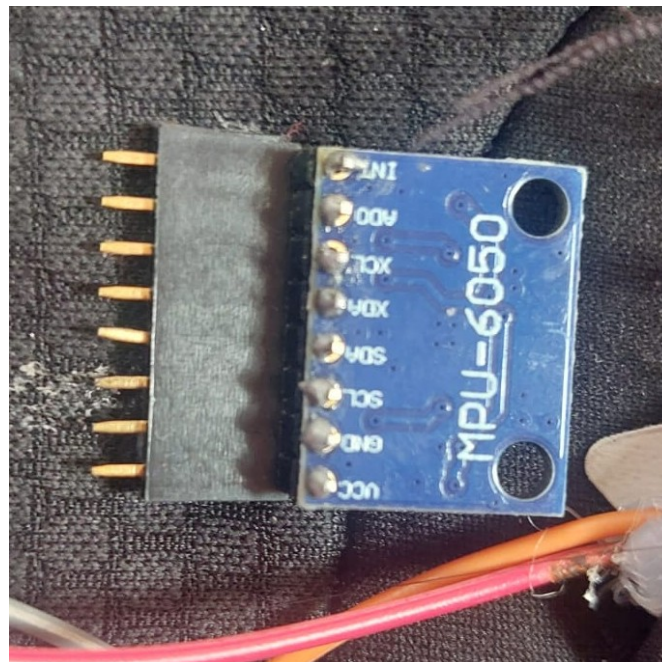


Figure 2.2: MPU6050 inertial measurement unit.

2.2.3 DFPlayer Mini

The DFPlayer Mini is a compact MP3 playback module that can store prerecorded audio files on a microSD card and receive playback commands through UART. In the prototype, it is paired with a **3 W speaker** for audible local output. The module can directly drive a small speaker, making it appropriate for a portable prototype. In the proposed system it serves as the primary local voice-output device.

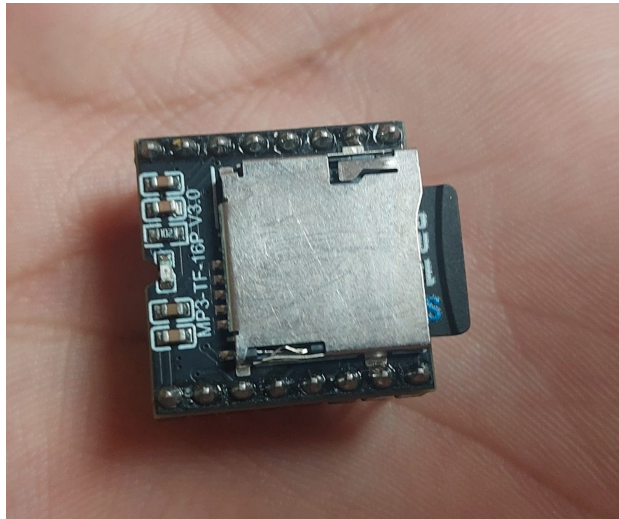


Figure 2.3: DFPlayer Mini audio module.

2.2.4 Copper-Tape Touch Detector (Touch Sensor)

Copper tape is used as a simple conductive touch-sensing surface at six locations on the glove. When a finger touches a copper-tape detector, the ESP32 reads a digital state indicating the contact. In the implemented logic, **1 represents untouched** and **0 represents touched**. The six touch states are active machine-learning inputs, not only control signals, and are combined with the five flex-sensor values and six MPU6050 values to form the 17-parameter feature vector.



Figure 2.4: Copper-Tape touch Detector.

2.2.5 3 W Speaker

The prototype uses a 3 W, 4-ohm speaker as the final audio-output transducer. The ESP32 sends the selected audio command to the DFPlayer Mini over UART, and the DFPlayer Mini plays the corresponding prerecorded MP3 file through the speaker. This provides local voice output without requiring a phone, computer, or internet connection.

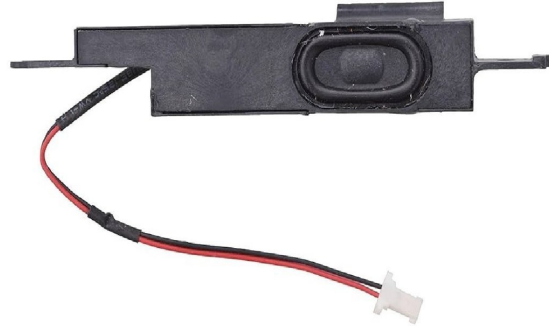


Figure 2.5: Speaker for audio output.

2.2.6 ESP32 Microcontroller

The ESP32 is the central controller of the glove. It acquires sensor data, manages communication with the MPU6050 and DFPlayer Mini, performs gesture inference, and controls the operating-mode logic. Its analog inputs support flex-sensor measurements, while I2C and UART provide interfaces to the IMU and audio module respectively.



Figure 2.6: ESP32 DevKit.

2.2.7 UART Communication

UART is an asynchronous serial communication method using transmit and receive lines. In this project, UART is used to send commands from the ESP32 to the DFPlayer Mini. After a gesture is recognized, the ESP32 determines the corresponding audio file and sends a playback command.

2.2.8 Random Forest for Gesture Recognition

Random Forest is an ensemble classifier that combines multiple decision trees. Each tree produces a class prediction and the final class is selected by majority voting. The method is robust to nonlinear feature relationships and can be effective for sensor-based classification [12].

For the final system design, each gesture sample is intended to contain 17 active input parameters:

- Flex Sensor 1–5 (five finger-bending parameters)
- Accelerometer X, Y, Z (three parameters)
- Gyroscope X, Y, Z (three parameters)
- Copper-tape Touch Sensor 1–6 (six touch-state parameters)

Thus, the Random Forest receives a 17-parameter sensor-fusion input consisting of five flex values, six MPU6050 motion values, and six touch-sensor values. The touch parameters are not merely interface controls; they directly participate in the gesture prediction process and can have a strong influence on the class selected by the forest.

The forest contains 25 decision trees with a maximum depth of 8 and a minimum of 2 samples per leaf. Gini impurity is used as the splitting criterion.

2.2.9 Gesture-to-Voice Conversion

Once a gesture has been classified, the corresponding output can be mapped to a prerecorded audio file. In alphabet mode, the recognized letters can be accumulated in a character buffer and used to form words. In Word Mode, a selected gesture can directly trigger a predefined phrase. The DFPlayer Mini performs the final audio playback.

3. Methodology

3.1 Data Collection

A labelled dataset was created from sensor readings generated while performing the 26 ASL alphabet gestures. The glove was worn by the participants, and each gesture was held while multiple consecutive sensor samples were recorded. The collected data were stored in CSV format together with the corresponding alphabet class label.

The updated system uses five flex sensors, six touch sensors, and an MPU6050 inertial measurement unit (IMU). The five flex sensors measure finger bending, while the six touch sensors detect contact between selected regions of the glove. The MPU6050 provides three-axis acceleration and three-axis angular velocity measurements. Together, these sensors provide 17 active parameters for gesture classification.

Flex sensors 1–5 correspond to the pinky, ring, middle, index, and thumb fingers, respectively. Touch sensors 1–6 correspond to the index, middle, ring, and pinky fingers, the upper side of the index finger, and the region between the index and middle fingers. The six touch channels are included directly in the machine-learning feature vector because their states provide additional information for distinguishing between gesture configurations.

The dataset contains labelled samples representing the ASL alphabet classes A–Z. Letters J and Z are represented using static approximations because their conventional ASL representations involve movement.

[illegible]

(a) Recorded sensor data

(b) Dataset sample

Figure 3.1: Dataset collection and sensor readings.

3.2 Data Preprocessing

The collected session-wise CSV files were processed before model training. The preprocessing stage consisted of data cleaning, validation, sensor normalization, feature-vector construction, and dataset preparation. The objective was to ensure that the final dataset contained valid, consistent, and correctly labelled sensor samples.

3.2.1 Data Cleaning and Validation

The raw serial datasets were inspected to identify and remove merged rows, duplicate headers, corrupted labels, incomplete records, and abnormal sensor readings caused by logger restarts or recording errors. Manual inspection was also performed to identify incorrectly labelled samples, duplicated records, and sensor values that did not correspond to the intended gesture.

The manually verified samples were checked against their corresponding gesture labels to ensure consistency between the recorded sensor values and the target class. Missing or invalid records were removed or repaired where appropriate. After validation, the sensor features were converted into numerical form and consolidated into a consistent dataset format.

For final model evaluation, a session-wise held-out testing protocol was used. The held-out session was excluded from model training and was used only for final evaluation, providing a more realistic estimate of generalization to unseen recording conditions than a random sample-level split.

3.2.2 Sensor Normalization

To maintain consistency between data collection, model training, and embedded deployment, the continuous sensor inputs were normalized before being provided to the classifier.

Each flex sensor was calibrated separately using an open-hand (flat) value and a fist-pose value. The normalized flex-sensor value was calculated as:

$$\text{flex_norm} = \text{clip} \left(\frac{\text{raw} - \text{flat}}{\text{fist} - \text{flat}}, 0, 1 \right) \quad (3.1)$$

where raw is the measured ADC value, flat is the open-hand calibration value, and fist is the fist-pose calibration value. The clipping operation restricts the normalized value to the range $[0, 1]$. If the calibration values are equal, a fallback normalized value of 0.5 is used.

For the MPU6050, each raw signed 16-bit accelerometer and gyroscope reading was normalized using:

$$\text{mpu_norm} = \frac{\text{raw_int16}}{32768.0} \quad (3.2)$$

This normalization was independently applied to a_x , a_y , a_z , g_x , g_y , and g_z . The six touch sensors are digital inputs and therefore retain binary values without additional scaling.

Although Random Forest is a tree-based algorithm and does not require feature scaling, the normalization process ensures that the same sensor representation is used consistently during calibration, dataset preparation, and embedded inference.

3.2.3 Final 17-Parameter Feature Vector

After preprocessing, the sensor readings were combined into a 17-parameter sensor-fusion feature vector. The feature vector consists of five flex-sensor parameters, six touch-sensor parameters, and six MPU6050 parameters.

Table 3.1: Final Sensor-Fusion Feature Vector

No.	Parameter	Type	Representation
1–5	Flex 1–5: Pinky, Ring, Middle, Index, Thumb	Analog	Normalized to $[0, 1]$
6–11	Touch 1–6: Index, Middle, Ring, Pinky, Upper Index, Index–Middle	Digital	Binary values
12–14	Accelerometer: a_x, a_y, a_z	IMU	Normalized signed values
15–17	Gyroscope: g_x, g_y, g_z	IMU	Normalized signed values

The resulting 17-parameter vector is used as the common feature representation for both Random Forest training and real-time ESP32 inference.

3.2.4 Sensor Parameter and Class Mapping

The physical sensor numbering and dataset column names were maintained consistently throughout data collection, model training, and embedded deployment.

Table 3.2: Final Sensor-Location and Dataset-Parameter Mapping

Type	No.	Dataset Column	Physical Location / Parameter
Flex	1	flex_pinky	Pinky finger
Flex	2	flex_ring	Ring finger
Flex	3	flex_middle	Middle finger
Flex	4	flex_index	Index finger
Flex	5	flex_thumb	Thumb
Touch	1	touch_index	Index finger
Touch	2	touch_mid	Middle finger
Touch	3	touch_ring	Ring finger
Touch	4	touch_pink	Pinky finger
Touch	5	touch_r	Upper side of index finger
Touch	6	touch_u	Between index and middle finger
MPU6050	–	ax, ay, az	Accelerometer X, Y, Z
MPU6050	–	gx, gy, gz	Gyroscope X, Y, Z

The gesture class labels correspond to the 26 alphabet classes A–Z used for Random Forest

classification.

3.3 Dataset Preparation

3.3.1 Session Data Merging

The sensor data were collected in multiple sessions to capture variations in hand position, gesture execution, and sensor behaviour. Each session was stored as an individual CSV file. After cleaning and validation, the valid session files were merged into a single consolidated dataset.

This session-wise approach provided an organized method for identifying individual recording sessions and allowed corrupted or low-quality sessions to be inspected and corrected before being included in the final dataset.

3.3.2 Final Dataset

The final consolidated dataset consisted of the 17 sensor features and their corresponding gesture labels. The resulting dataset was used as the input for Random Forest model development and evaluation. Figure 3.2 presents the overall data training pipeline, from session-wise data collection and preprocessing to Random Forest training, evaluation, and model export.

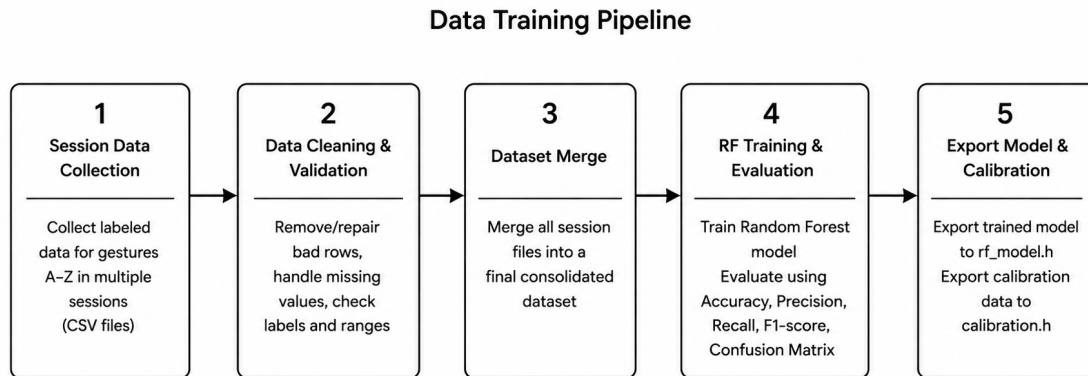


Figure 3.2: Data training and model development pipeline.

3.4 Random Forest Model Training and Evaluation

For final model evaluation, a session-wise held-out testing protocol was used. The collected data were organized according to their recording sessions, and approximately 20% of the complete sessions were reserved for testing, with at least one complete session held out. The remaining sessions were used for model training. Thus, samples from the held-out session(s) were not used during model training. This approach reduces the possibility of highly similar samples from the same recording session appearing in both the training and testing sets and provides a

more realistic estimate of model generalization to unseen recording conditions.

The Random Forest classifier was trained using Python and the scikit-learn library. Random Forest was selected because it can perform multi-class classification using multiple decision trees and is suitable for sensor-based gesture recognition.

The model parameters used during training were:

- Number of trees: 25
- Maximum tree depth: 8
- Minimum samples per leaf: 2
- Maximum features per split: sqrt
- Splitting criterion: Gini impurity
- Class weighting: balanced_subsample
- Random seed: 42
- Number of input parameters: 17
- Number of output classes: 26

The trained model was evaluated using accuracy, precision, recall, F1-score, and a confusion matrix. These metrics were used to assess the classification performance of the model across the 26 gesture classes.

If the model performance was found to be unsatisfactory, weak or incorrectly represented gestures were identified and additional data were collected to improve the dataset before re-training.

3.5 Embedded Implementation

After successful model training and evaluation, the trained Random Forest model was prepared for deployment on the ESP32.

3.5.1 Model Export

The trained Random Forest model was exported to a C/C++ header file named `rf_model.h`. A custom Python conversion process was used to serialize the decision-tree structure and enable direct tree-traversal inference on the ESP32. The flex-sensor calibration parameters were separately exported to `calibration.h`. These parameters allow the embedded system to apply the same flex-sensor normalization process used during data preparation.

3.5.2 ESP32 Integration

The ESP32 acquires five flex-sensor values, six touch-sensor states, and six MPU6050 parameters. These readings are processed to construct the same 17-parameter feature vector used during model training.

The feature vector is then passed to the embedded Random Forest classifier for real-time gesture prediction. A majority-vote smoothing approach is applied across consecutive predictions to reduce unstable single-sample outputs.

3.5.3 Real-Time Gesture Inference

During real-time operation, the ESP32 continuously reads the sensors, performs the required preprocessing, constructs the 17-parameter feature vector, and performs Random Forest inference. The decision logic applies stability, confidence or margin checks, and a cooldown mechanism to prevent unreliable or repeated predictions from generating unwanted outputs.

3.6 Operating Modes and Audio Output

The system supports two operating modes controlled by the mode push button.

3.6.1 Alphabet Mode and Mapping

In alphabet mode, each recognized gesture is mapped to one of the 26 ASL alphabet letters. The recognized letters can be accumulated in a character buffer to form words according to the implemented interaction logic. The corresponding prerecorded audio file is selected through the alphabet mapping.

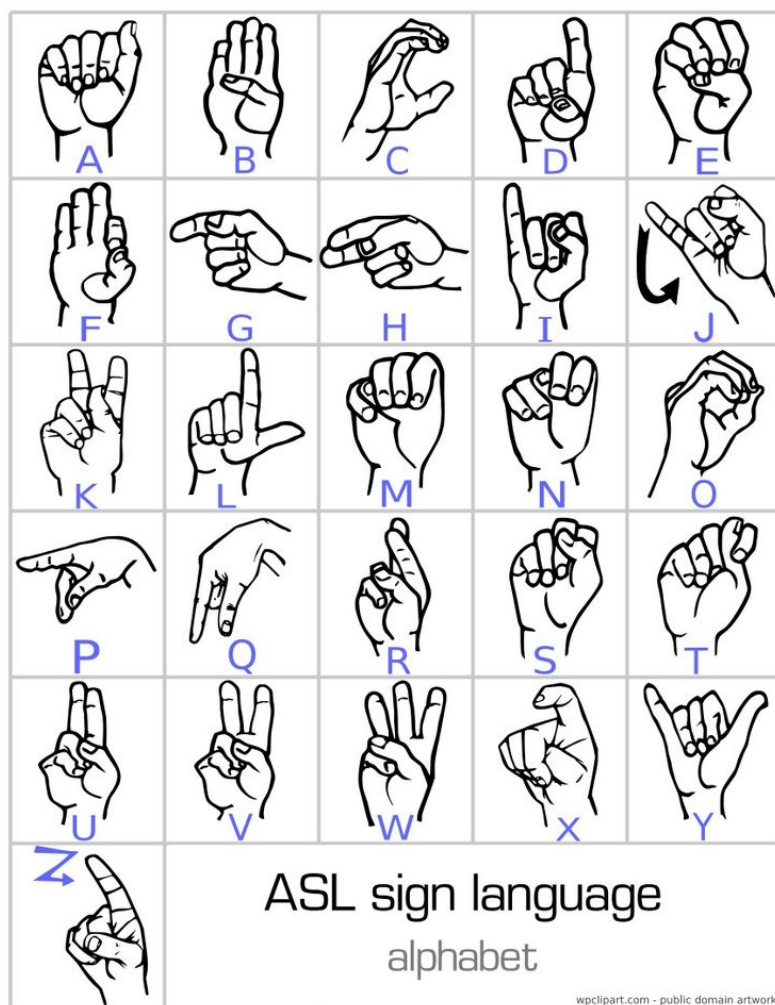


Figure 3.3: ASL alphabet Categorization.

Table 3.3: ASL Alphabet Gesture-to-Audio Mapping

Mode	Alphabet	MP3 Code
Alphabet	A	0001
Alphabet	B	0002
Alphabet	C	0003
Alphabet	D	0004
Alphabet	E	0005
Alphabet	F	0006
Alphabet	G	0007
Alphabet	H	0008
Alphabet	I	0009
Alphabet	J	0010
Alphabet	K	0011
Alphabet	L	0012
Alphabet	M	0013
Alphabet	N	0014
Alphabet	O	0015
Alphabet	P	0016
Alphabet	Q	0017
Alphabet	R	0018
Alphabet	S	0019
Alphabet	T	0020
Alphabet	U	0021
Alphabet	V	0022
Alphabet	W	0023
Alphabet	X	0024
Alphabet	Y	0025
Alphabet	Z	0026

3.6.2 Word Mode Gesture Mapping

Word Mode was implemented to provide faster communication by directly mapping selected ASL alphabet gestures to predefined words or phrases. Unlike Alphabet Mode, where each

recognized gesture represents an individual alphabet letter, Word Mode uses a selected set of alphabet gestures as shortcuts for commonly used expressions.

Only twelve alphabet gestures are assigned to Word Mode outputs: K, H, F, D, V, B, Y, W, I, R, G, and T. Each mapped gesture is associated with a predefined word or phrase and a corresponding MP3 file stored on the microSD card of the DFPlayer Mini. The mapping used in the implemented system is presented in Table 3.4.

Table 3.4: Word Mode Gesture-to-Audio Mapping

Mode	Alphabet	Word/Phrase	MP3 Code
Word	K	Hello	0027
Word	H	Go	0028
Word	F	Good	0029
Word	D	Bad	0030
Word	V	Thank you	0031
Word	B	Stop	0032
Word	Y	Call me	0033
Word	W	I need water	0034
Word	I	Can I go Toilet	0035
Word	R	Promise	0036
Word	G	Just a bit	0037
Word	T	Sorry	0038

When Word Mode is selected using the mode-selection push button, the recognized alphabet class is first checked against the predefined Word Mode mapping. If the recognized letter is one of the twelve mapped classes, the corresponding MP3 code is selected and sent to the DFPlayer Mini for playback. The associated prerecorded word or phrase is then played through the connected speaker.

The remaining alphabet classes are intentionally left unmapped in Word Mode. Therefore, when an unmapped alphabet gesture is recognized while Word Mode is active, no word or phrase is triggered and the system remains silent. This prevents unintended audio outputs from gestures that have not been assigned to a predefined shortcut.

3.6.3 Audio Output

The ESP32 sends the appropriate playback command to the DFPlayer Mini based on the selected alphabet or word mapping. The DFPlayer Mini retrieves the corresponding prerecorded audio file from the microSD card and plays it through the connected speaker.

3.7 Development and Deployment Workflow

The practical workflow used for developing and deploying the gesture-recognition system is organized as follows:

- i. Connect the glove to the ESP32 and verify the five flex sensors, six touch sensors, MPU6050, and DFPlayer Mini wiring. Insert the microSD card into the DFPlayer Mini.
- ii. Upload the data-collection firmware to the ESP32 and maintain the serial communication rate at 115200 baud.
- iii. Calibrate the five flex sensors using the PC-side calibration procedure. Store the calibration values in a timestamped JSON file.
- iv. Collect labelled A–Z gesture samples using the latest calibration values. Save each recording session as a CSV file containing sensor readings and class labels.
- v. Repeat calibration and data collection across multiple sessions where required to improve robustness against sensor drift, hand position, and user variation.
- vi. Clean and validate the individual session files by removing invalid records, correcting or removing corrupted data, and verifying the gesture labels.
- vii. Merge the cleaned session files into a single consolidated dataset.
- viii. Prepare the 17-parameter feature vectors and train the Random Forest model.
- ix. Evaluate the trained model using accuracy, precision, recall, F1-score, and confusion matrix.
- x. Export the trained Random Forest model to `rf_model.h` and the sensor calibration parameters to `calibration.h`.
- xi. Integrate the exported model and calibration parameters into the ESP32 deployment firmware.
- xii. Upload the completed firmware to the ESP32 and perform real-time gesture classification.
- xiii. Verify alphabet and word-mode mappings and confirm that the recognized gestures trigger the correct DFPlayer Mini audio output.

4. Hardware Configuration / Experimental Setup

4.1 Hardware Components

The final prototype consists of the following principal hardware components:

- Cotton glove
- Five flex sensors mounted along the five fingers
- ESP32 DevKit V1
- MPU6050 six-axis IMU providing $a_x, a_y, a_z, g_x, g_y, g_z$
- Six copper-tape touch detectors used as active digital gesture features
- DFPlayer Mini with microSD card
- 3 W, 4-ohm speaker
- Two 3.7 V Li-ion batteries
- L7805CV3 5 V linear voltage regulator
- Push button for Alphabet/Word mode selection
- Perfboard / prototype wiring

4.2 Hardware Assembly

Five flex sensors were mounted on the five fingers of the glove: pinky, ring, middle, index, and thumb. The sensors were positioned along the fingers so that their resistance changed according to finger bending.

The MPU6050 was positioned on the back of the hand to capture three-axis acceleration and three-axis angular velocity. Six copper-tape touch detectors were positioned on the index, middle, ring, and pinky fingers, the upper side of the index finger, and the region between the index and middle fingers. These touch sensors provide additional digital features for gesture classification.

The flex sensors are connected to the ESP32 ADC inputs through voltage-divider circuits. The pinky, ring, middle, and index flex sensors use 200k Ω resistors, while the thumb flex sensor uses a 120k Ω resistor.

The MPU6050 communicates with the ESP32 through the I²C interface, with SDA connected to GPIO21 and SCL connected to GPIO22. The six touch sensors are connected to GPIO26, GPIO23, GPIO19, GPIO18, GPIO25, and GPIO27. The DFPlayer Mini communicates with the ESP32 through UART using GPIO16 and GPIO17 and drives the connected 3 W speaker.

The prototype is powered by two 3.7 V Li-ion batteries connected as the implemented battery supply. An L7805CV3 linear voltage regulator provides the nominal 5 V supply required by the corresponding circuit sections. The ESP32 development board provides the 3.3 V logic supply, and all circuit sections share a common ground. No external capacitors are installed in the implemented prototype.

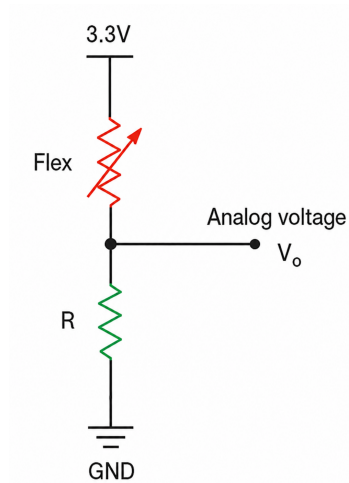


Figure 4.1: Voltage-divider circuit for the flex sensor.

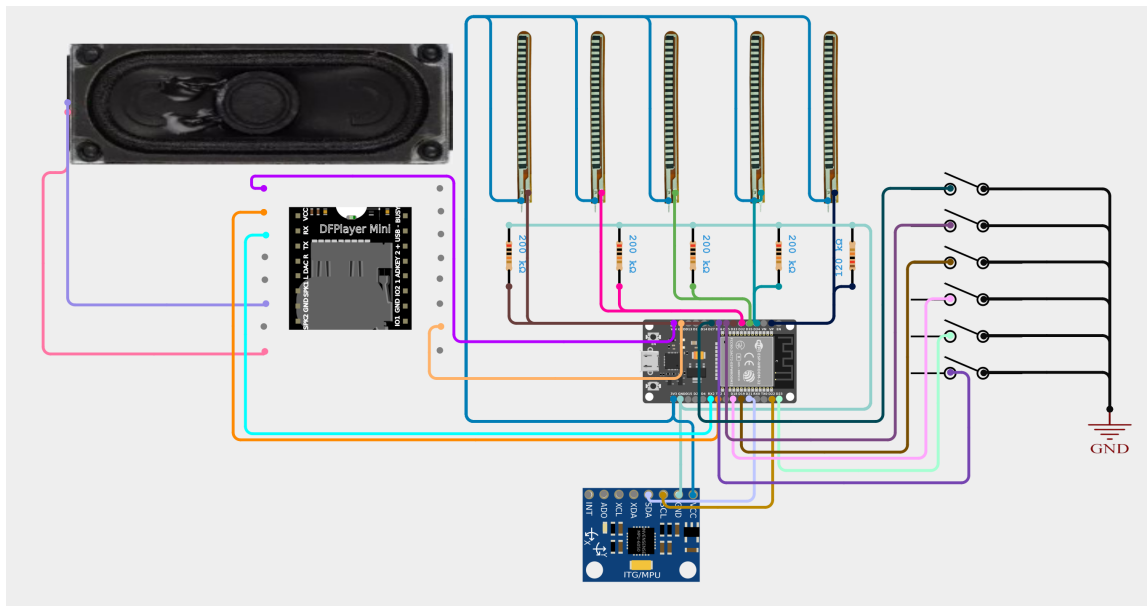


Figure 4.2: Final circuit prototype.

4.2.1 Circuit and Power Configuration

The implemented power configuration uses two 3.7 V Li-ion cells as the battery supply and an L7805CV3 TO-220 linear voltage regulator to obtain a nominal 5 V rail. The regulator is the only dedicated external voltage regulator used in the prototype.

The 5 V supply is distributed to the circuit components requiring this voltage, while the ESP32 development board provides the required 3.3 V logic supply. The sensor and communication circuits share a common ground to provide a common electrical reference.

The power and circuit configuration shown in Figure 4.3 represents the physical implementation of the final prototype. No additional capacitor components are included in the documented circuit because they were not installed in the implemented prototype.

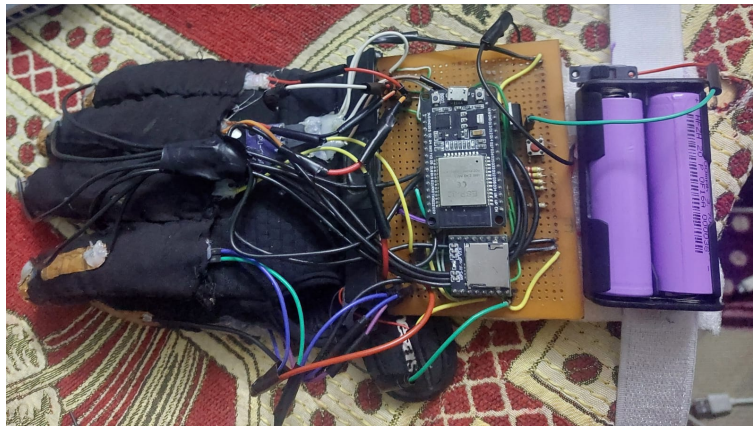
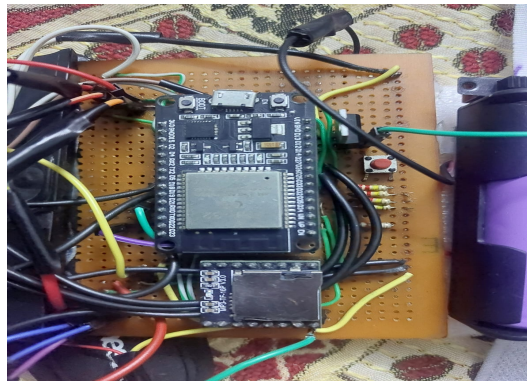


Figure 4.3: Final hardware circuit prototype.



(a) Final wiring

Figure 4.4: Hardware assembly of the glove system.

4.3 Experimental Setup

The experimental setup consisted of the completed sensorized glove, ESP32 development board, MPU6050 IMU, six touch detectors, DFPlayer Mini, speaker, battery supply, and associated wiring. The glove was worn by the participants during data collection and real-time testing.

Before each experimental session, the sensor connections and power supply were checked. The flex sensors were calibrated using the established calibration procedure, and the ESP32 was initialized with the required firmware. The sensor positions were maintained consistently during the experiments to reduce variations caused by changes in sensor placement.

The experimental setup was used for two main purposes: collection of labelled gesture data for machine-learning development and evaluation of the final prototype during real-time gesture recognition.

4.4 Experimental Procedure

The experimental procedure was carried out in the following sequence:

- i. The sensorized glove was worn, and the sensors were positioned consistently on the hand.
- ii. The glove was connected to the ESP32, and the connections of the five flex sensors, six touch sensors, MPU6050, DFPlayer Mini, and speaker were verified.
- iii. The system was powered on, and the ESP32 and connected peripherals were initialized.
- iv. The five flex sensors were calibrated using the established open-hand and fist-position calibration procedure.
- v. Each ASL alphabet gesture was performed and held consistently while the ESP32 recorded the corresponding sensor readings.
- vi. The recording process was repeated for all required alphabet gestures and across multiple sessions to obtain representative sensor data.
- vii. The recorded sensor measurements and corresponding gesture labels were stored as session-wise CSV files.
- viii. The collected sessions were inspected to verify the sensor readings, gesture labels, and completeness of the recorded data.
- ix. The prepared dataset and trained Random Forest model were used for the final embedded implementation as described in Chapter 3.
- x. The deployment firmware and exported Random Forest model were uploaded to the ESP32.
- xi. Real-time gesture recognition was performed by continuously acquiring the 17 sensor parameters and passing them to the embedded classifier.
- xii. Prediction smoothing and decision logic were applied to obtain stable gesture predictions.
- xiii. Both Alphabet Mode and Word Mode were tested using the mode-selection push button.

- xiv. The recognized gestures were verified to produce the corresponding alphabet letters, words, or predefined phrases.
- xv. The DFPlayer Mini playback commands were verified, and the corresponding prerecorded audio was played through the connected speaker.
- xvi. The final system observations and classification results were recorded for subsequent performance analysis.

5. System design

5.1 System Architecture

The overall system consists of sensing, processing, decision, and output stages. Flex sensors capture finger-bending information, while the MPU6050 supplies hand-motion and orientation information. The ESP32 collects and processes these signals, performs gesture classification, and applies the selected operating mode. The DFPlayer Mini then plays the corresponding prerecorded audio.

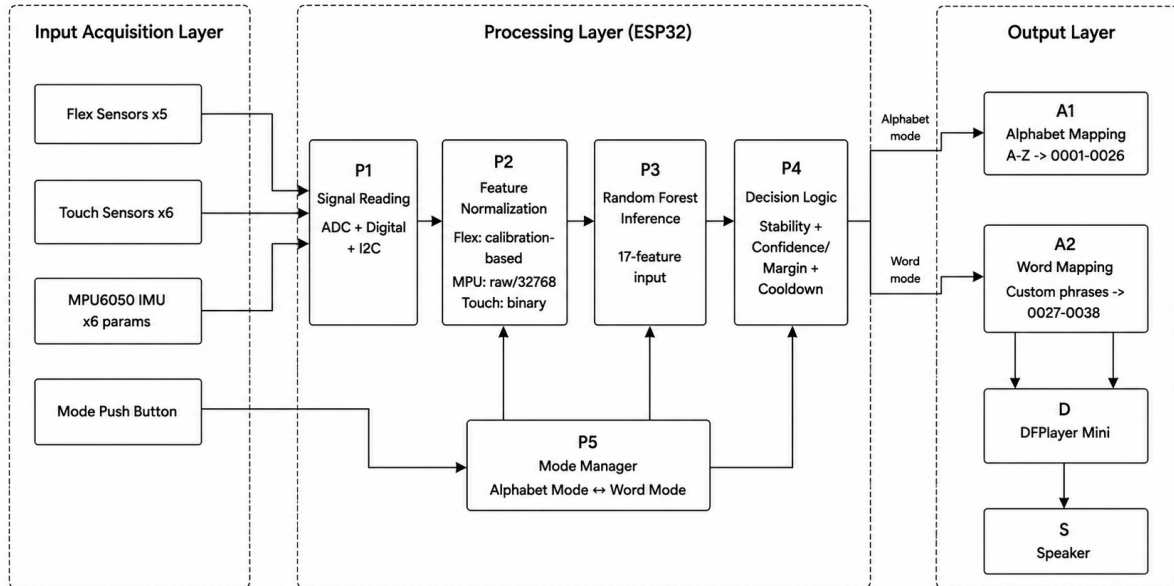


Figure 5.1: Overall system block diagram.

5.1.1 Block Diagram Description

The overall operation of the gesture recognition glove can be explained as follows:

- i. **Sensor Input:** The glove uses five flex sensors to measure finger bending, six touch sensors to detect finger contact, and an MPU6050 IMU to measure three-axis acceleration and three-axis gyroscope values. A mode push button is also provided to switch between alphabet mode and word mode.
- ii. **Signal Reading:** The ESP32 collects the sensor signals through the appropriate interfaces. The flex sensors are read using ADC inputs, the touch sensors provide digital states, and the MPU6050 communicates with the ESP32 through the I²C interface.

- iii. **Feature Normalization:** The acquired sensor values are converted into a consistent format before classification. The flex-sensor readings are normalized using their calibration values, the MPU6050 readings are scaled from their raw range, and the touch-sensor readings are represented as binary values.
- iv. **Sensor-Fusion Feature Vector:** The normalized readings are combined to form a 17-feature sensor-fusion vector consisting of five flex-sensor parameters, six MPU6050 parameters, and six touch-sensor parameters. This feature vector is used as the input to the machine-learning classifier.
- v. **Random Forest Inference:** The 17-feature input vector is processed by the embedded Random Forest model running on the ESP32. The classifier uses multiple decision trees and determines the recognized gesture based on the combined voting of the trees.
- vi. **Decision Logic:** The ESP32 applies decision logic to the classifier output. Stability checking, confidence or margin checking, and a cooldown mechanism are used to prevent unstable or repeated predictions from producing unwanted outputs.
- vii. **Mode Management:** The mode manager uses the mode push button to select between alphabet mode and word mode. The selected mode determines how the recognized gesture is interpreted and mapped to an audio output.
- viii. **Alphabet Mode:** In alphabet mode, the recognized gesture is mapped to a corresponding alphabet letter from A to Z. Each alphabet gesture is associated with a predefined audio file identifier ranging from 0001 to 0026.
- ix. **Word Mode:** In word mode, recognized gestures are mapped to predefined words or custom phrases. The corresponding audio files are assigned identifiers ranging from 0027 to 0038.
- x. **Audio Playback:** After the appropriate alphabet or word mapping is selected, the ESP32 sends the corresponding playback command to the DFPlayer Mini. The DFPlayer Mini retrieves the selected prerecorded audio file and sends the audio signal to the connected speaker.
- xi. **Output:** The speaker produces the prerecorded letter, word, or phrase corresponding to the recognized hand gesture, providing audible communication to the user.

5.2 System Operation Flowchart

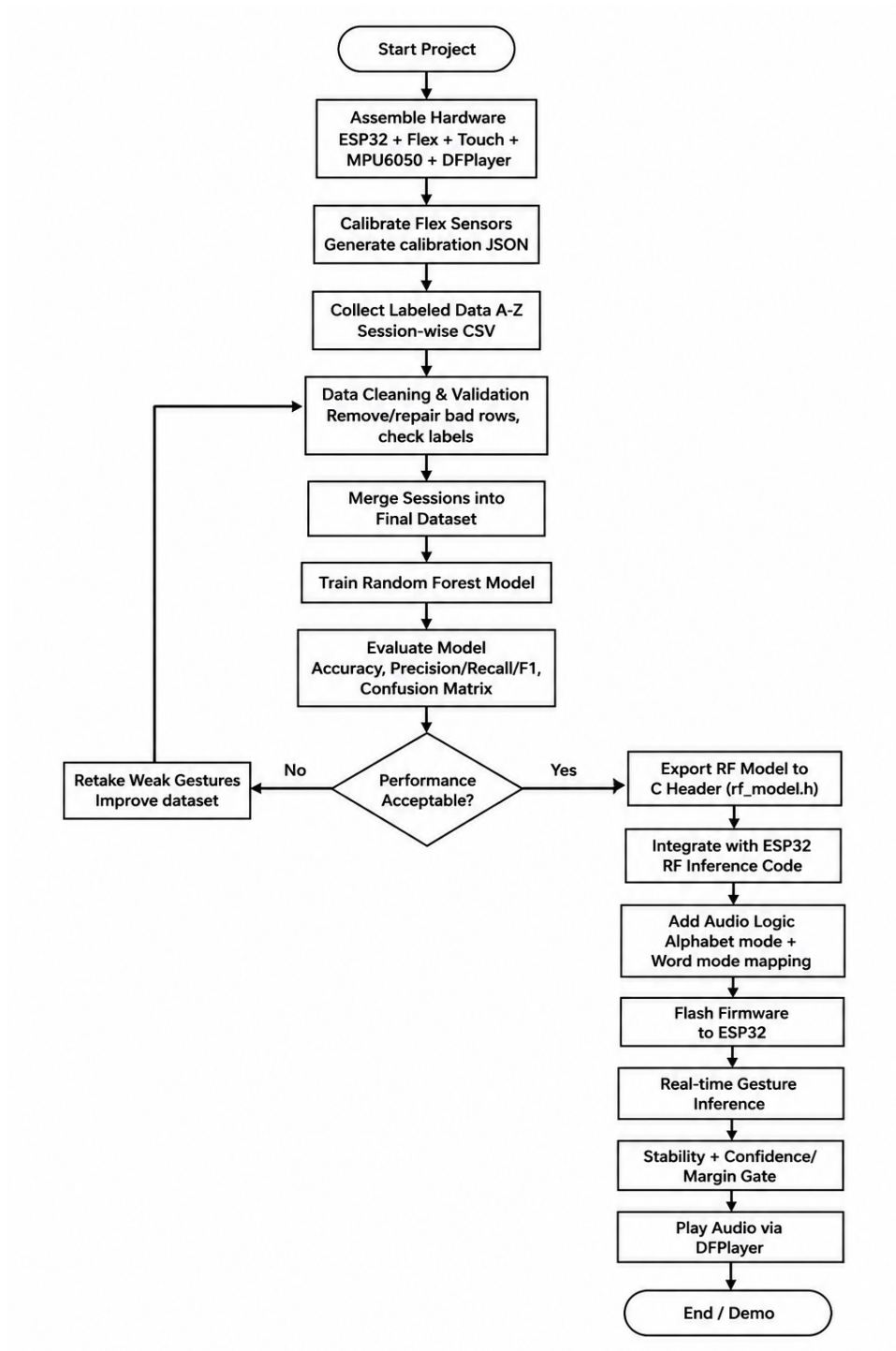


Figure 5.2: Overall gesture recognition and audio output flowchart.

The operational flow is:

- i. Start the project and assemble the complete hardware system, including the ESP32, five flex sensors, six touch sensors, MPU6050, and DFPlayer Mini.
- ii. Calibrate the flex sensors and generate the calibration JSON file for accurate sensor readings.
- iii. Collect labeled gesture data for alphabets A–Z and save the readings as session-wise CSV files.
- iv. Perform data cleaning and validation by removing or repairing invalid rows and checking the correctness of the gesture labels.
- v. Merge the cleaned session-wise data into a single final dataset for machine-learning training.
- vi. Train the Random Forest classifier using the final sensor-fusion dataset.
- vii. Evaluate the trained model using accuracy, precision, recall, F1-score, and a confusion matrix.
- viii. Determine whether the model performance is acceptable. If the performance is not acceptable, retake weak or incorrectly recognized gestures, improve the dataset, and repeat the data-cleaning and training process.
- ix. If the performance is acceptable, export the trained Random Forest model into a C header file (rf_model.h) for deployment on the ESP32.
- x. Integrate the exported Random Forest model with the ESP32 inference code for real-time gesture recognition.
- xi. Add the audio-processing logic, including alphabet mode and word-mode gesture mapping.
- xii. Flash the completed firmware onto the ESP32.
- xiii. Perform real-time gesture inference by continuously reading the sensor data and classifying the detected hand gestures.
- xiv. Apply stability and confidence-margin checks to ensure that only reliable and stable predictions are accepted.
- xv. Send the corresponding audio command to the DFPlayer Mini and play the prerecorded audio through the speaker.
- xvi. Complete the system operation and demonstrate the final working prototype.

6. Results & Discussion

6.1 Experiment Setup

The final prototype used for testing consisted of a sensorized glove integrated with an ESP32 DevKit V1, five flex sensors, six copper-tape touch channels, an MPU6050 IMU, a DFPlayer Mini, a 3 W speaker, two 3.7 V Li-ion batteries, and an L7805CV3 5 V regulator. During experiments, the regulator output was connected to ESP32 VIN, and all modules shared a common ground.

The sensor configuration used in the experiments included:

- Flex channels (5): pinky, ring, middle, index, thumb
- Touch channels (6): index, middle, ring, pinky, R, and U
- MPU6050 channels (6): $a_x, a_y, a_z, g_x, g_y, g_z$

Thus, the deployed model operated on a 17-parameter feature vector.

A total of 26 gesture classes (A–Z) were tested. The cleaned dataset used for model development contained 17,600 labelled samples collected across multiple sessions. The final Random Forest model was evaluated using a session-wise held-out testing protocol. Complete recording session(s) were excluded from training and reserved exclusively for testing. This ensured that the final performance measurement was obtained from sensor samples belonging to sessions that were not used during model training.

Testing was performed under practical glove-use conditions, including repeated gesture holds and real-time prediction with audio output. The final prototype photographs and circuit documentation presented in Chapter 4 define the physical arrangement used to obtain the results reported in this chapter.

6.2 Software Development Approach

The software pipeline was implemented as an end-to-end workflow from data acquisition to embedded deployment:

1. Sensor acquisition and session-wise data logging from ESP32
2. Flex calibration and normalization parameter generation
3. Dataset cleaning, quality filtering, and session merging
4. Random Forest training and held-out session evaluation

5. Model export to embedded C/C++ header format
6. Real-time ESP32 inference with DFPlayer-based audio output

Data-processing software: Python scripts were used for calibration, data collection, merge/clean operations, strict filtering, Random Forest training, and report-plot generation. Cleaning steps addressed malformed values, noisy/corrupted readings, and class imbalance handling as required. The recorded dataset interface shown in Figure 3.1 provides visual evidence of the sensor-data collection and labelling process.

Random Forest implementation: The classifier was implemented using scikit-learn with a lightweight embedded-friendly configuration. The input vector contained 17 parameters (5 flex + 6 touch + 6 MPU6050).

Prediction smoothing: In firmware, predictions were stabilized using temporal consistency checks, confidence/margin gating, and cooldown timing before issuing audio output, reducing unstable frame-to-frame outputs.

Mode-selection logic: A push button (GPIO12) toggled between:

- **Alphabet mode:** direct A–Z output
- **Word mode:** mapped phrases for selected letters

Alphabet and word mapping: Alphabet mode used standard indexed audio mapping, while word mode used a custom letter-to-phrase mapping according to the project vocabulary list.

DFPlayer communication: ESP32 communicated with DFPlayer Mini over UART (TX2/RX2), and audio files were addressed by indexed track numbers in the SD-card MP3 folder.

ESP32 firmware architecture: The firmware handled sensor reading, normalization, Random Forest traversal inference, mode logic, and audio playback control in a compact real-time loop.

No separate graphical user interface was implemented on the final glove. The user interacts with the system through the physical glove and mode-selection button, while feedback is provided through the DFPlayer Mini and speaker. The recorded sensor-data interface shown in Figure 3.1 is included as software-side evidence of the data-collection process.

6.3 Model Deployment

The final trained Random Forest model was exported to an embedded header file (`rf_model.h`) containing serialized decision-tree node structures and traversal logic compatible with ESP32 firmware execution.

Deployment steps:

1. Train RF model on cleaned dataset in Python
2. Export model into `rf_model.h`

3. Integrate `rf_model.h` and `calibration.h` into ESP32 firmware
4. Run real-time 17-feature inference on-device
5. Map predictions to DFPlayer audio output

Calibration integration: Flex normalization in deployment used calibration constants generated from pre-session flat/fist measurements, ensuring consistency between data collection and live inference.

Memory/resource perspective: A compact RF configuration was selected to satisfy ESP32 flash constraints while preserving accuracy. Model size and firmware footprint were monitored during compilation and iterative tuning.

Deployment result: The model was successfully deployed on ESP32, and real-time gesture recognition with DFPlayer audio output worked in both Alphabet and Word modes.

6.4 Testing/evaluation/validation

6.4.1 Model Performance

The Random Forest classifier was evaluated using a held-out session to assess its ability to generalize to sensor data that were not used during model training. The overall performance of the classifier is presented in Figure 6.1.

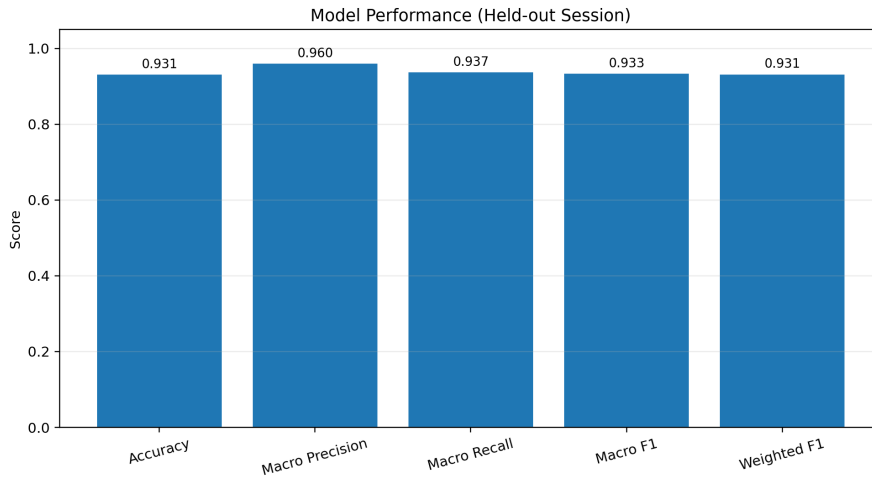


Figure 6.1: Overall performance of the Random Forest model on the held-out session.

The classifier achieved an overall accuracy of 93.1%. The macro precision was 96.0%, while the macro recall was 93.7%. The resulting macro F1-score was 93.3%, and the weighted F1-score was 93.1%. These results demonstrate that the Random Forest classifier provides strong overall recognition performance across the 26 alphabet classes.

The high macro precision indicates that the model generally produces reliable class predictions, while the high macro recall indicates that most of the actual gesture classes are successfully detected. The close values of macro F1-score and weighted F1-score further indicate that

the model performs relatively consistently across the evaluated classes, although some individual gestures remain more difficult to classify.

6.4.2 Random Forest Feature Importance

Feature importance was analyzed to determine the relative contribution of the 17 sensor parameters used by the Random Forest classifier. The results are shown in Figure 6.2.

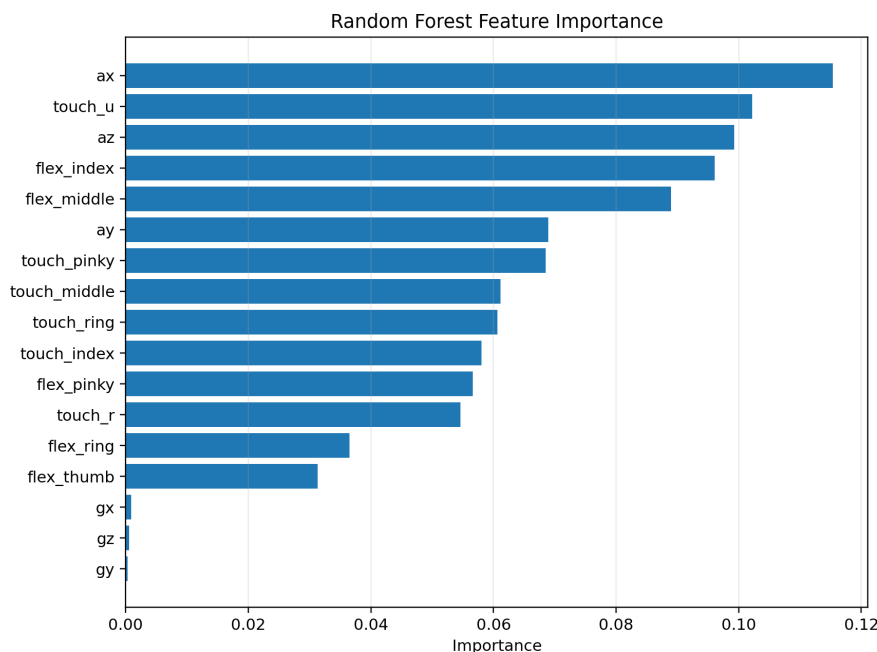


Figure 6.2: Feature importance of the 17 sensor parameters in the Random Forest model.

The feature-importance analysis shows that the accelerometer X-axis (ax) is the most influential feature in the trained Random Forest model. The upper touch sensor (touch_u) and accelerometer Z-axis (az) are the next most important features. Among the flex sensors, the index and middle finger sensors (flex_index and flex_middle) provide relatively strong contributions to classification.

The touch sensors also make a significant contribution to gesture recognition. In particular, touch_u, touch_pinky, touch_middle, _ring, and touch_index have noticeable feature-importance values. This demonstrates that the six copper-tape touch sensors are active and useful classification features rather than being used only as control inputs.

In contrast, the gyroscope features (gx, gy and gz) have very low importance compared with the accelerometer, touch, and flex features. This indicates that, for the evaluated dataset, accelerometer information, touch states, and finger-bending measurements contributed more strongly to the classification decisions than the gyroscope measurements.

6.4.3 Training Behaviour

The effect of the number of decision trees on the Random Forest model was investigated. Figure 6.3 presents the training accuracy and held-out session accuracy for different values of the

n_estimators parameter.

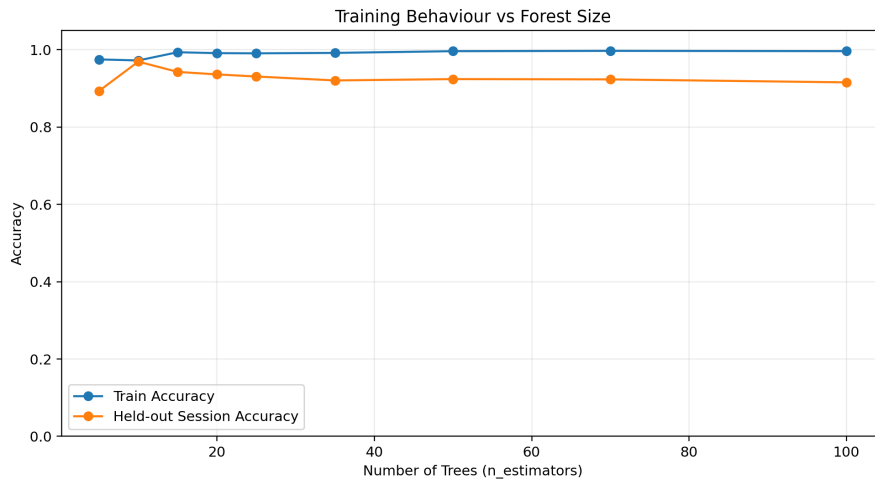


Figure 6.3: Training and held-out session accuracy for different Random Forest sizes.

The training accuracy remains very high as the number of trees increases and approaches approximately 99–100%. The held-out session accuracy is lower than the training accuracy for most forest sizes, indicating a generalization gap between the training data and unseen data.

The held-out session shows the final 25-tree configuration achieved the reported overall deployment accuracy of 93.1%. Although a different number of trees produced the highest held-out accuracy in the comparative experiment, 25 trees were retained for the final implementation as a practical balance between classification performance and the computational requirements of deployment on the ESP32. The reported 93.1% result therefore corresponds specifically to the final deployed 25-tree configuration and its held-out evaluation.

6.4.4 Per-Class Performance

The precision, recall, and F1-score for each of the 26 alphabet classes are presented in Figure 6.4.

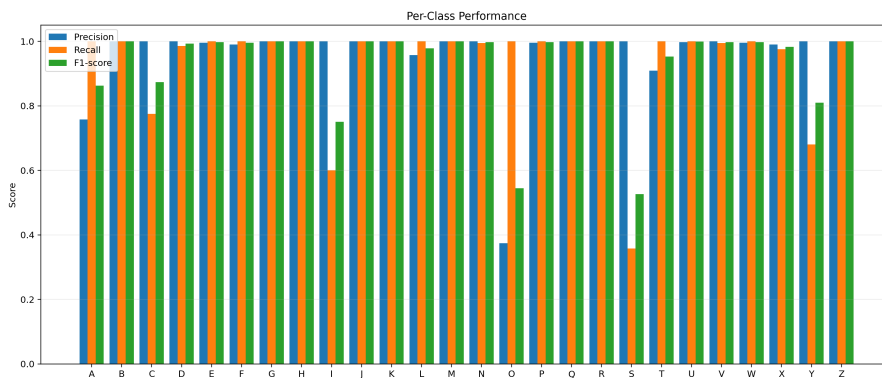


Figure 6.4: Precision, recall, and F1-score for individual alphabet classes.

The per-class results show that most alphabet classes achieve high precision, recall, and F1-scores, with many classes reaching values close to 1.0. This indicates that the Random Forest

model can reliably distinguish a large proportion of the static alphabet gestures.

However, some classes show comparatively lower performance. The most noticeable reductions can be observed for classes such as **I**, **O**, and **S**. The classes **A**, **C**, and **Y** also show comparatively lower F1-scores than the strongest-performing classes. These differences indicate that certain gestures produce sensor patterns that are more difficult for the classifier to distinguish.

The variation in per-class performance demonstrates that overall accuracy alone does not completely describe the behaviour of the multi-class gesture-recognition system. Individual finger positions, sensor placement, hand posture, touch relationships, and similarities between different alphabet configurations can influence classification performance.

6.4.5 Confusion Matrix Analysis

The confusion matrix was generated from predictions on the held-out test session(s) presented in Figure 6.5. The matrix provides a detailed view of the correctly and incorrectly classified alphabet gestures. The diagonal elements represent correctly classified gestures, while the off-diagonal elements indicate misclassifications between gesture classes.

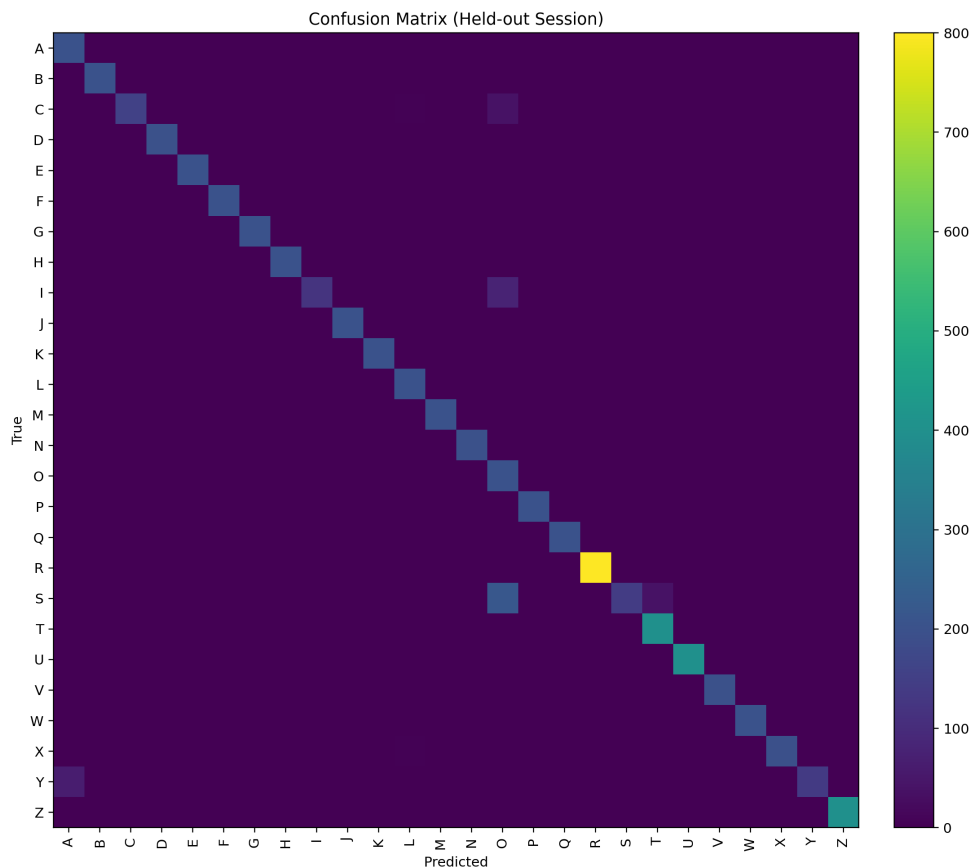


Figure 6.5: Confusion matrix of the Random Forest classifier on the held-out session.

The confusion matrix shows a strong diagonal pattern across most of the 26 classes. This indicates that a large proportion of the test samples were correctly classified into their cor-

responding alphabet classes. The strong diagonal is consistent with the overall accuracy of 93.1%.

A smaller number of off-diagonal predictions can also be observed. Some noticeable confusion occurs between **C** and **O**, and between **I** and **O**. Additional misclassifications are visible around the **S**, **T**, and **U** classes. Some errors can also be observed among neighbouring classes toward the **T–Z** region. These errors suggest that some gestures produce similar combinations of finger position, touch state, and hand orientation.

The majority of the remaining classes show strong diagonal responses with limited off-diagonal activity. Therefore, although some individual gestures remain challenging, the Random Forest provides effective multi-class classification for the overall alphabet dataset.

6.4.6 Discussion

The experimental results demonstrate that the Random Forest classifier can effectively recognize glove-based hand gestures using the combined 17-parameter sensor feature vector. The model achieved an overall accuracy of 93.1%, with a macro precision of 96.0%, macro recall of 93.7%, macro F1-score of 93.3%, and weighted F1-score of 93.1% on the held-out session.

The feature-importance analysis demonstrates the usefulness of combining different sensor types. Accelerometer and touch-sensor features make major contributions to the classification process, while the flex sensors provide important information about individual finger positions. The relatively low importance of the gyroscope channels suggests that the evaluated dataset relies more strongly on hand orientation, finger bending, and touch relationships than on angular-velocity information.

The training-behaviour analysis shows that training accuracy remains close to 100%, while held-out session accuracy is lower. This difference indicates a degree of overfitting and highlights the importance of evaluating the model using unseen sessions rather than relying only on training accuracy.

Overall, the results demonstrate that the Random Forest classifier is suitable for real-time gesture recognition on the ESP32. The high held-out performance indicates that the proposed sensor-fusion approach can effectively distinguish most of the 26 alphabet gestures. Nevertheless, improvements in dataset diversity, sensor placement, calibration, and dynamic gesture representation could further improve the recognition of the weaker classes and increase robustness across different users and operating conditions.

7. Conclusions

This project investigated the design and implementation of a wearable gesture-based voice communication system using glove-mounted sensors and embedded machine learning. The system integrates five flex sensors, six MPU6050 motion parameters, six copper-tape touch sensors, ESP32 processing, and DFPlayer Mini audio playback into a low-cost assistive communication platform.

A complete data-processing pipeline was established from sensor acquisition and data cleaning to model training, evaluation, conversion, and embedded deployment. The Random Forest classifier uses multiple decision trees to classify the 17-parameter sensor-fusion input. The trained model was converted into C/C++ logic and integrated with the ESP32 for on-device inference.

The model achieved an overall accuracy of 93.1% on the held-out session, with a macro precision of 96.0%, macro recall of 93.7%, macro F1-score of 93.3%, and weighted F1-score of 93.1%. These results demonstrate the effectiveness of combining flex-sensor, touch-sensor, and MPU6050 measurements for multi-class gesture recognition.

The feature-importance analysis showed that the accelerometer X-axis, upper touch sensor, accelerometer Z-axis, index flex sensor, and middle flex sensor were among the most influential features. The touch sensors also contributed significantly to classification, confirming their usefulness as active inputs to the gesture-recognition model. In comparison, the gyroscope features had relatively low importance in the evaluated dataset.

The confusion matrix demonstrated a strong diagonal pattern for most alphabet classes, while the per-class analysis showed that some gestures, particularly I, O, S, A, C, and Y, were comparatively more difficult to classify. These differences highlight the influence of gesture similarity, sensor placement, hand posture, and individual sensor variation on recognition performance.

The project therefore provides a practical foundation for a standalone gesture-to-voice communication device capable of performing gesture recognition directly on an ESP32. Future improvements can focus on collecting larger and more diverse datasets, improving sensor placement and calibration, adding temporal information for dynamic gestures such as J and Z, and improving the robustness of the system across different users and hand configurations.

8. Limitations and Future Enhancement

8.1 Limitations

- i. Although the model demonstrates strong classification performance on the available dataset, real-time prediction can still be affected by sensor noise, glove-fit variation, and session-to-session sensor drift.
- ii. Some gesture classes may remain difficult to distinguish because of similarities in their sensor patterns. In addition, the dynamic letters J and Z are represented using static approximations rather than their complete temporal movements.
- iii. The system was evaluated using the available user and session data; therefore, broader multi-user and multi-session testing is still required to establish robustness for wider real-world deployment.

8.2 Future Enhancements

- i. Collect a larger multi-user dataset with repeated sessions and additional samples for gesture classes that show higher confusion or lower classification performance.
- ii. Incorporate sequence-based processing, temporal segmentation, or filtering to better represent dynamic gestures such as J and Z and to further reduce unstable predictions.
- iii. Improve deployment robustness through adaptive sensor calibration, further validation of Word Mode mappings, and compact PCB integration for improved wearable usability.

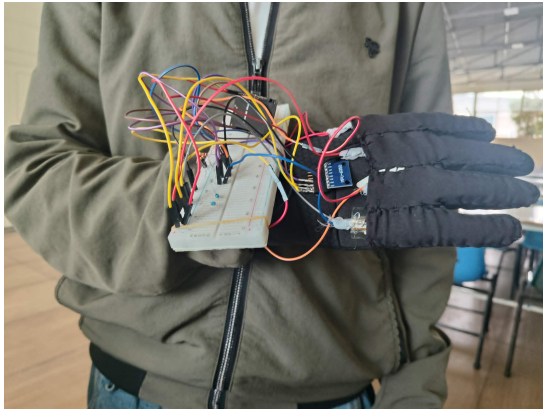
References

- [1] E Spandana, M Rajasekar, and N Sandhya. Care-giver alerting for bedridden patients using hand gesture recognition system. *Journal of Physics: Conference Series*, 1921(1):012077, May 2021.
- [2] Chandraman M, Santhiyakumari N, Ganesh Venkateshwaran S, Damodharan M, Santhiya C, and Subalakshimi V. Echogesture communication: Gesture-based systems for individuals with disabilities. *Journal of Electronics and Informatics*, 6(3):253–261, Sep 2024.
- [3] Sangeeta Kurundkar. Hand gesture recognizer smart glove using esp32 and mpu 6050. *International Journal for Research in Applied Science and Engineering Technology*, 12(11):641–645, Nov 2024.
- [4] Anna Filipowska, Wojciech Filipowski, Paweł Raif, Marcin Pieniżek, Julia Bodak, Piotr Ferst, Kamil Pilarski, Szymon Sieciński, Rafał Jan Doniec, Julia Mieszczanin, Emilia Skwarek, Katarzyna Bryzik, Maciej Henkel, and Marcin Grzegorzec. Machine learning-based gesture recognition glove: Design and implementation. *Sensors*, 24(18):6157, Sep 2024.
- [5] Amos Matsiko. Tracking hand movements with a smart glove. *Science Robotics*, 9(87), Feb 2024.
- [6] Wen-xiao Lu, Ping Fang, Ming-lu Zhu, Yi-run Zhu, Xin-jian Fan, Tian-chen Zhu, Xuan Zhou, Feng-xia Wang, Tao Chen, and Li-ning Sun. Artificial intelligence-enabled gesture–language–recognition feedback system using strain–sensor–arrays–based smart glove. *Advanced Intelligent Systems*, 5(8), Apr 2023.
- [7] Mengmeng Qin. Enhancing gesture recognition systems: integrating deep learning with advanced flexible sensing technologies. In Yang Yue, editor, *International Conference on Optics, Electronics, and Communication Engineering (OECE 2024)*, page 47. SPIE, Nov 2024.
- [8] Arvin Tashakori, Zenan Jiang, Amir Servati, Saeid Soltanian, Harishkumar Narayana, Katherine Le, Caroline Nakayama, Chieh-ling Yang, Z. Jane Wang, Janice J. Eng, and Peyman Servati. Capturing complex hand movements and object interactions using machine learning-powered stretchable smart textile gloves. *Nature Machine Intelligence*, 6(1):106–118, Jan 2024.

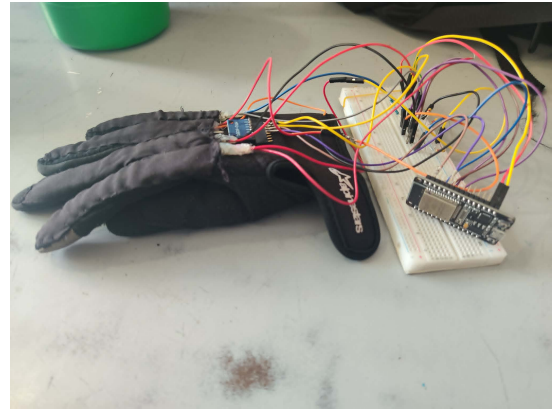
- [9] Shixiang Lin, Chengfeng Sun, Yuhang Huang, Bin Cheng, Heng Luo, Minglu Zhu, Tao Chen, Fengxia Wang, and Lining Sun. Machine learning-assisted intelligent gesture recognition and vr space communication smart-glove system using strain sensors. *Advanced Materials Technologies*, Dec 2025.
- [10] B. Sarada. Smart glove for gesture recognition using iot. *INTERNATIONAL JOURNAL OF SCIENTIFIC RESEARCH IN ENGINEERING AND MANAGEMENT*, 09(04):1–9, Apr 2025.
- [11] Unnathi E, A. K Sreeja, Ravi Teja, and Vinutha LV. Real-time sign language recognition and translation with glove and mobile integration. In *2024 Second International Conference on Advances in Information Technology (ICAIT)*, page 1–6. IEEE, Jul 2024.
- [12] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

Appendix:

1.Additional Hardware Photographs:

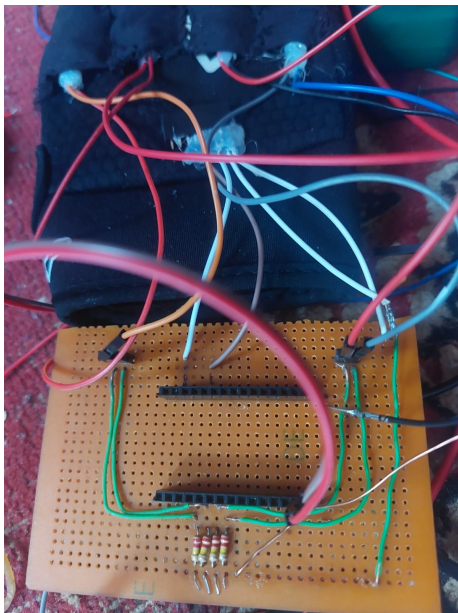


(a) Prototype photograph

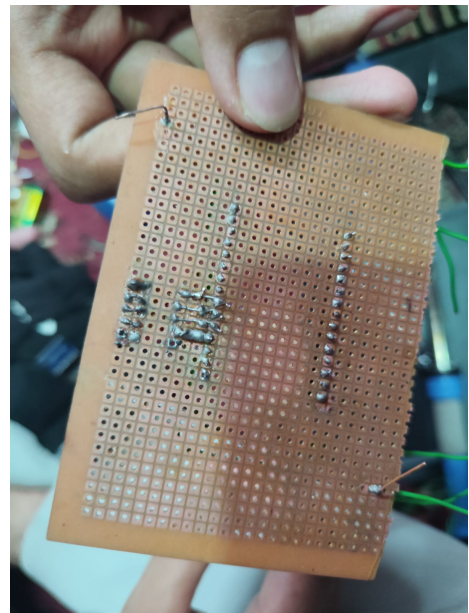


(b) Prototype photograph

Figure 8.1: Additional prototype photographs.



(a) Soldered assembly



(b) Component wiring

Figure 8.2: Additional hardware assembly photographs.